

Workshop 1: Introduction to scikit-learn

TECH6

Richard Foltyn
NHH Norwegian School of Economics

October 15, 2026

See GitHub repository for notebooks and data:

<https://github.com/richardfoltyn/TECH6-H26>

Exercise 1: Polynomial under- and overfitting

Consider the following non-linear model,

$$y_i = f(x_i) + \epsilon_i$$
$$f(x) = \cos\left(\frac{3}{2}\pi x\right)$$
$$\epsilon_i \stackrel{\text{iid}}{\sim} N(0, \sigma_\epsilon^2)$$

where y_i is a trigonometric function of x_i but is measured with an additive error ϵ_i . In this exercise, we are going to approximate y_i using polynomials in x_i of varying degrees.

Part 1 — Creating and plotting a sample

The function `fcn()` implements the true relationship $y = f(x)$.

```
[1]: import numpy as np

def fcn(x):
    """
    True function without errors.
    """
    return np.cos(1.5 * np.pi * x)
```

Using this function, create a sample of size $N = 50$ where the x_i are randomly drawn from a uniform distribution on the interval $[0, 1]$ and $\sigma_\epsilon = 0.2$ (initialize your RNG with a seed of 1234). Then generate y_i according to the equation given above.

Create a scatter plot of the sample (x_i, y_i) and add a line depicting the true relationship without measurement error.

Part 2 — Polynomial approximations

In order to approximate this relationship using polynomials, you first want to visualize how the polynomial degree affects the model fit. To this end, complete the template for `plot_poly_degrees()` below to create a figure with 6 subplots, each showing the predicted values for a polynomial of degree d , where

$d \in \{0, 1, 2, 3, 10, 15\}$. Each panel should additionally show the sample scatter plot and the true function $y = f(x)$.

How does the quality of the approximation change as you increase d ? Do higher-order polynomials always perform better?

Hints:

- You should build a pipeline, e.g., using `make_pipeline()` which combines the `PolynomialFeatures` transformation and `LinearRegression` to approximate y as a polynomial in x .
- When creating polynomials with `PolynomialFeatures(..., include_bias=True)` you need to fit the model *without* an additional intercept as the intercept is already included in the polynomial.

```
[4]: import matplotlib.pyplot as plt

def plot_poly_degrees(degrees, X, y):
    """
    Fit and plot polynomial regression models of different degrees.

    Parameters
    -----
    degrees : array-like
        List of polynomial degrees to fit and plot.
    X : array-like
        Explanatory variable.
    y : array-like
        Response variable.
    """

    # Create figure with 3 columns
    ncol = 3
    nrow = int(np.ceil(len(degrees) / ncol))

    fig, axes = plt.subplots(
        nrow, ncol, figsize=(8, 5), sharex=True, sharey=True, constrained_layout=True
    )

    # x-values for predicting & plotting
    xvalues = np.linspace(0.0, 1.0, 101)
    # True y-values
    y_true = fcn(xvalues)

    for i, ax in enumerate(axes.flatten()):
        # Degree for current sub-plot
        d = degrees[i]

        # TODO: Create polynomial transformation + estimation pipeline

        # TODO: Fit the model to the sample data

        # TODO: Predict the response variable for the x-values

        # TODO: Plot predicted values

        # Plot true relationship
        ax.plot(xvalues, y_true, color='black', lw=1.0)
        # Plot sample as scatter plot
        ax.scatter(X, y, s=20, color='none', edgecolor='steelblue', lw=0.75)
        ax.set_ylim((-1.5, 1.5))
        ax.legend()
```

Part 3 — Train-test split and polynomial evaluation

In this part, we evaluate how well polynomials of degrees $d = 0, 1, \dots, 15$ fit the data and predict unseen observations using a train-test split.

First, use `train_test_split()` from `sklearn.model_selection` to split the sample (X, y) from Part 1 into training (70%) and test (30%) subsets, setting `random_state=1234`.

Next, complete the template for `compute_poly_mse()` below. For each degree d , the function should: 1. Build a pipeline combining `PolynomialFeatures(degree=d, include_bias=True)` and `LinearRegression(fit_intercept=False)`. 2. Fit the pipeline on the training data $(X_{\text{train}}, y_{\text{train}})$. 3. Predict the response on both the training and test sets. 4. Compute the Mean Squared Error (MSE) on each subset using `mean_squared_error()`. 5. Return two NumPy arrays containing the training and test MSEs.

```
[7]: from sklearn.metrics import mean_squared_error
     from sklearn.model_selection import train_test_split

     # TODO: Split sample into training (70%) and test (30%) sets
     # X_train, X_test, y_train, y_test = ...

     def compute_poly_mse(degrees, X_train, y_train, X_test, y_test):
         """
         Compute training and test MSE for a sequence of polynomial degrees.
         """
         mse_train = []
         mse_test = []

         for d in degrees:
             # TODO: Build pipeline, fit on training data, compute MSEs
             pass

         return np.array(mse_train), np.array(mse_test)
```

Part 4 — Optimal polynomial degree and error curves

Using `compute_poly_mse()`, compute the training and test MSEs for a sequence of polynomial degrees $d = 0, 1, \dots, 15$.

Plot the training MSE and test MSE against d . Use `np.argmin()` to determine which polynomial degree results in the lowest test MSE.

How do the training and test errors evolve as the degree increases?

Part 5 — Plotting the fitted model

Re-estimate the model on the training set using the optimal polynomial degree found in Part 4. Create a plot showing: - The true relationship $y = f(x)$ - The training observations - The test observations (using a distinct marker/color) - The fitted polynomial curve evaluated on a dense grid $x \in [0, 1]$